

Performance Report: Constant-Time AES-128 Implementation

Group 5

May 8, 2026

Krish Arora
Jaykant Nayak
Ravi Arora
Pranav Krishna
Joel Bansal

1 Introduction

The objective of this assignment was to implement a fully functional, constant-time version of the Advanced Encryption Standard (AES-128). Standard software implementations of AES, such as the default OpenSSL implementation, rely heavily on precomputed lookup tables (S-boxes and T-tables) to accelerate the SubBytes and MixColumns transformations. While computationally efficient, these table lookups introduce data-dependent memory accesses, making the implementation highly vulnerable to cache-timing side-channel attacks.

To mitigate this, our implementation completely avoids lookup tables. Instead, it utilizes a bitsliced architecture leveraging the Boyar-Peralta 128-gate logic circuit to compute Galois Field inverses algebraically within the CPU registers. By utilizing SIMD Within A Register (SWAR) techniques, we process the state matrix using purely bitwise operations, achieving perfectly constant-time execution.

This report evaluates the performance overhead of our secure implementation compared to the heavily optimized, table-based OpenSSL reference implementation.

2 Performance Results

The performance was profiled by measuring the Total Clock Cycles and the average Cycles Per Instruction (CPI). Memory access times were modeled at 5 clock cycles. Following are the performance results as tested on csews65

Table 1: Naive approach

Metric	OpenSSL (Reference)	Custom	Ratio (Custom/OpenSSL)
Total Cycles	2,095,673	4270942	2.0263

Table 2: Final optimized results

Metric	OpenSSL (Reference)	Custom (Bitsliced)	Ratio (Custom/OpenSSL)
Total Cycles	2,095,673	2,107,181	1.0055
CPI	2.5001	2.4838	NA
Instructions	838236	848370	NA

3 Analysis and Discussion

3.1 Minimal Performance Overhead

Typically, replacing memory-fast lookup tables with algebraic logic circuits incurs a significant performance penalty. Our implementation bridges this gap through several critical optimizations:

- **Global Key Caching:** The expanded round keys are bitsliced and cached during the first execution. Subsequent encryptions using the same key bypass the expensive transposition and packing phase.
- **Loop Unrolling:** The 10 rounds of AES-128 are fully unrolled in the C code, eliminating loop control overhead and branch misprediction penalties.
- **SWAR Processing:** By packing the transposed 16-byte state into the lower 16 bits of 32-bit integers, the CPU can execute bitwise logical operations across the entire AES state simultaneously.

3.2 Security vs. Performance Trade-off

The OpenSSL implementation achieves its baseline performance by trading security for speed. Our implementation successfully eliminates this leakage vector.

4 Conclusion

The provided bitsliced AES-128 implementation successfully satisfies the constraint of zero secret-dependent memory accesses. It passes all functional correctness vectors and memory-trace analyses. Based on the performance evaluation, it maintains a highly competitive

execution profile, representing an optimal balance between cryptographic security and software performance.